

第十次理论作业



第十五章 1,4,5,6

P381

1、针对4种不同的指令集架构：

(1) 写出完成器-寄存器指令集架构可以使用的通用寄存器均为运算所需的指令序列, 其中寄存器-内存和寄存, 共8个寄存器。

(2) 假设与内存访问有关的指令均消耗3个时钟周期, 其他指令均消耗1个时钟周期, 计算完成不同指令序列消耗的时钟周期。

指令序列：

时钟周期：

栈式	累加器	寄存器-内存式	寄存器-寄存器式	访存指令：7条	访存指令：11条	访存指令：5条	访存指令：1条
PUSH A	LOAD B	LOAD R0,B	LOAD R0,A	其他指令：5条	其他指令：0条	其他指令：1条	其他指令：0条
PUSH B	ADD C	ADD R0,C,R0	LOAD R1,B	周期数：26	周期数：33	周期数：16	周期数：16
PUSH C	STORE X	MUL R0,A,R1	LOAD R2,B				
ADD	LOAD D	DIV D,R0,R2	LOAD R3,D				
MUL	DIV X	SUB R1,R2,R0	ADD R1,R2,R1				
PUSH D	STORE X	STORE R0,C	MUL R0,R1,R0				
PUSH B	LOAD B		DIV R3,R1,R3				
PUSH C	ADD C		SUB R0,R3,R0				
ADD	MUL A		STORE R0,X				
DIV	SUB X						
SUB	STORE X						
POP X							

4、以如下 C 语言程序为例，结合程序运行栈和其他地址空间的特点，说明如果将静态变量 `ss` 分配在程序运行栈中，程序运行的结果是什么？如果将 `ss` 分配在静态数据区，程序运行的结果是什么？并说明原因。

```
void foo() {
    static int ss = 0;
    printf("%d\n", ss++);
}

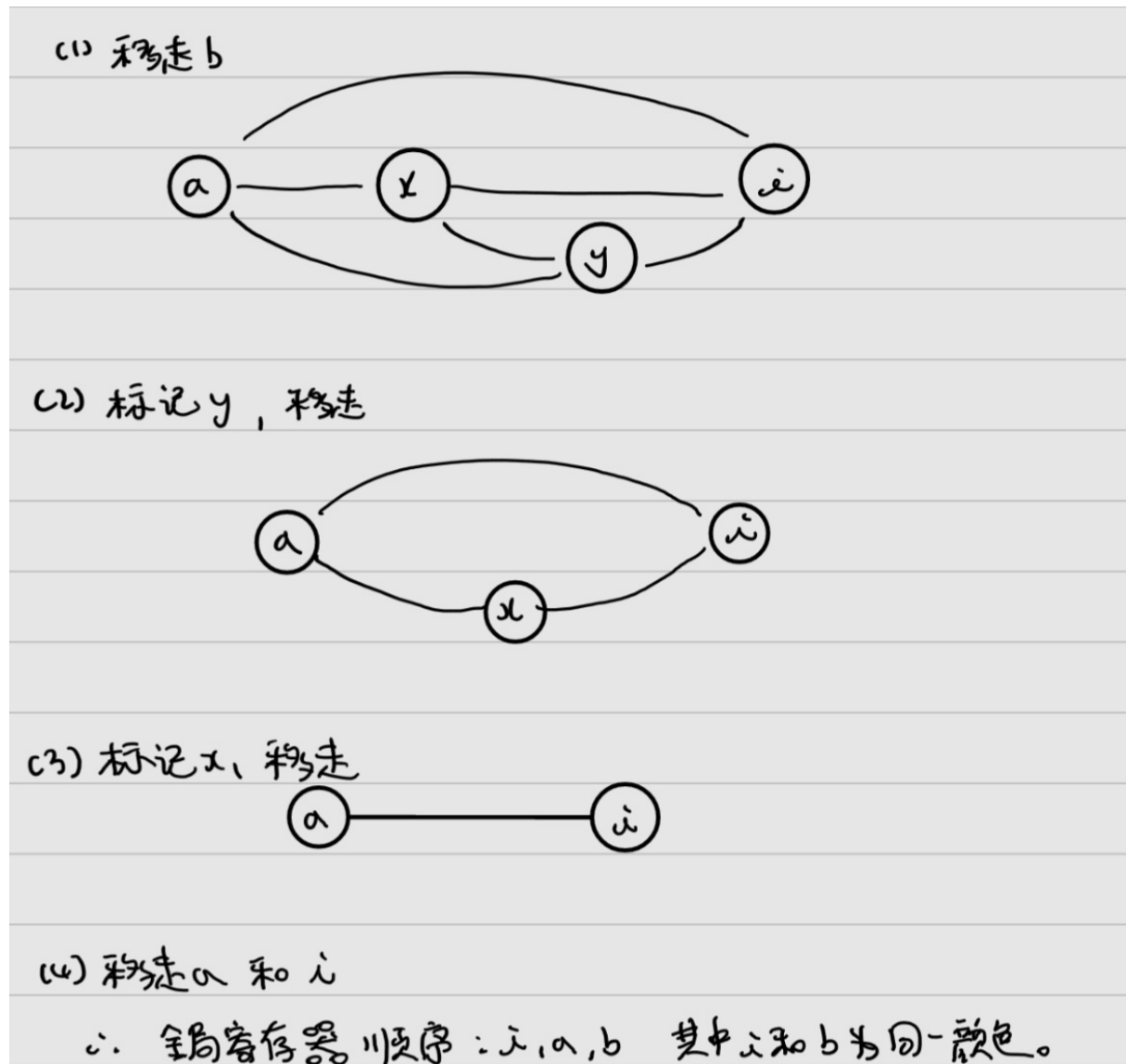
int main() {
    foo();
    foo();
}
```

如果

`ss` 被分配在程序运行栈中，每次调用 `foo()` 时都会重新初始化为 0，因此输出结果始终为 0。而如果 `ss` 被分配在静态数据区，由于 `static` 关键字的作用，它只会在程序运行期间初始化一次，并在函数调用之间保持值不变。

因此，第一次调用 `foo()` 输出 0，第二次调用输出 1。

5、给定全局变量冲突图如下，假设全局寄存器数量为2，请写出根据图着色算法为该图分配全局寄存器的过程和结果。



6、将表达式 $A \times (B+C) - D \times (B+C)$ 采用四元式进行表达，并生成相应 X86 汇编代码。假设其中变量 D 分别对应运行栈上的内存单元 ESP+10H, ESP+14H, ESP+18H, ESP+1CH，可用临时寄存器为 EAX, EDX, ECX，临时寄存器池的初始状态为空。

四元式表示

将表达式 $A \times (B+C) - D \times (B+C)$ 拆解为四元式：

1. $t1 = B+C$

2. $t2=A \times t1$
3. $t3=D \times t1$
4. $t4=t2-t3$

X86 汇编代码

```
; Step 1: t1 = B + C
MOV EAX, [ESP+14H]    ; 将 B 加载到 EAX
ADD EAX, [ESP+18H]    ; EAX = B + C, 即 t1 的值

; Step 2: t2 = A * t1
MOV EDX, [ESP+10H]    ; 将 A 加载到 EDX
IMUL EAX, EDX          ; EAX = A * t1, 即 t2 的值

; 保存 t2 的值到临时位置
PUSH EAX               ; 将 t2 压栈

; Step 3: t3 = D * t1
MOV ECX, [ESP+1CH]    ; 将 D 加载到 ECX
IMUL ECX, [ESP+14H]    ; ECX = D * t1, 即 t3 的值

; Step 4: t4 = t2 - t3
POP EAX               ; 将 t2 弹栈回 EAX
SUB EAX, ECX           ; EAX = t2 - t3, 即 t4 的值
```